

# Data Science for Economists

## Lecture 11: Complex Models-Trees and Neural Nets

---

Jiaxi Li

University of North Carolina | ECON 370

# Table of contents

1. Introduction
2. Trees
3. Bagging, Boosting and Random Forest
4. Basic Neural Networks with Torch
5. Torch Applications and GenAI

# Introduction

---

# Motivation

In Lectures 9 and 10, we worked with simpler models—mostly those with linear structure.

Today, we move beyond linearity and explore two more flexible models:

**Decision Trees** and **Neural Networks**.

The focus of this lecture is about the **structure of the models** and some related/improved modeling techniques.

Once you understand the model, we still do:

- the 3 steps in model fitting
- cross-validation for hyperparameter selection
- testing set for model evaluation

Note: Many materials in this lecture are drawn from Sections 8 and 10 of [Statistical Learning with R](#).

# Motivation

These represent **two fundamentally different ways** of letting a machine learn.

A helpful analogy comes from how we understand human cognition in psychology:

- **Trees - “Stereotype” Learning (Macro Approach)**
  - The model groups observations into broad categories based on their features.
  - Each group is then assigned a simple prediction rule—or even just a constant.
  - (This is intuitive to human understanding. So the interpretation of Trees are clear.)
- **Neural Networks - “Brain-like” Learning (Micro Approach)**
  - The model learns by passing information through layers of interconnected “neurons.”
  - Each neuron processes small pieces of information, and together they produce the final prediction.
  - (Just like how the brain aggregates tiny signals into one decision! e.g. Guess how much this pen is?)

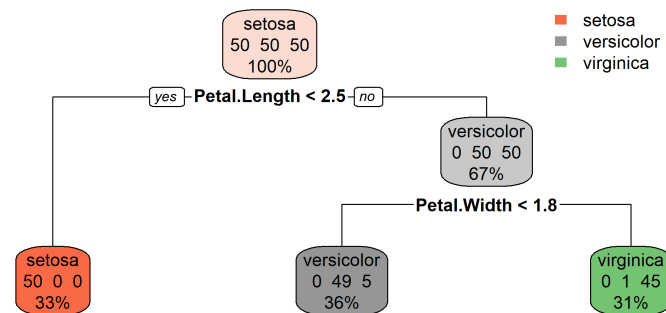
# Trees

---

# Why Called Tree?

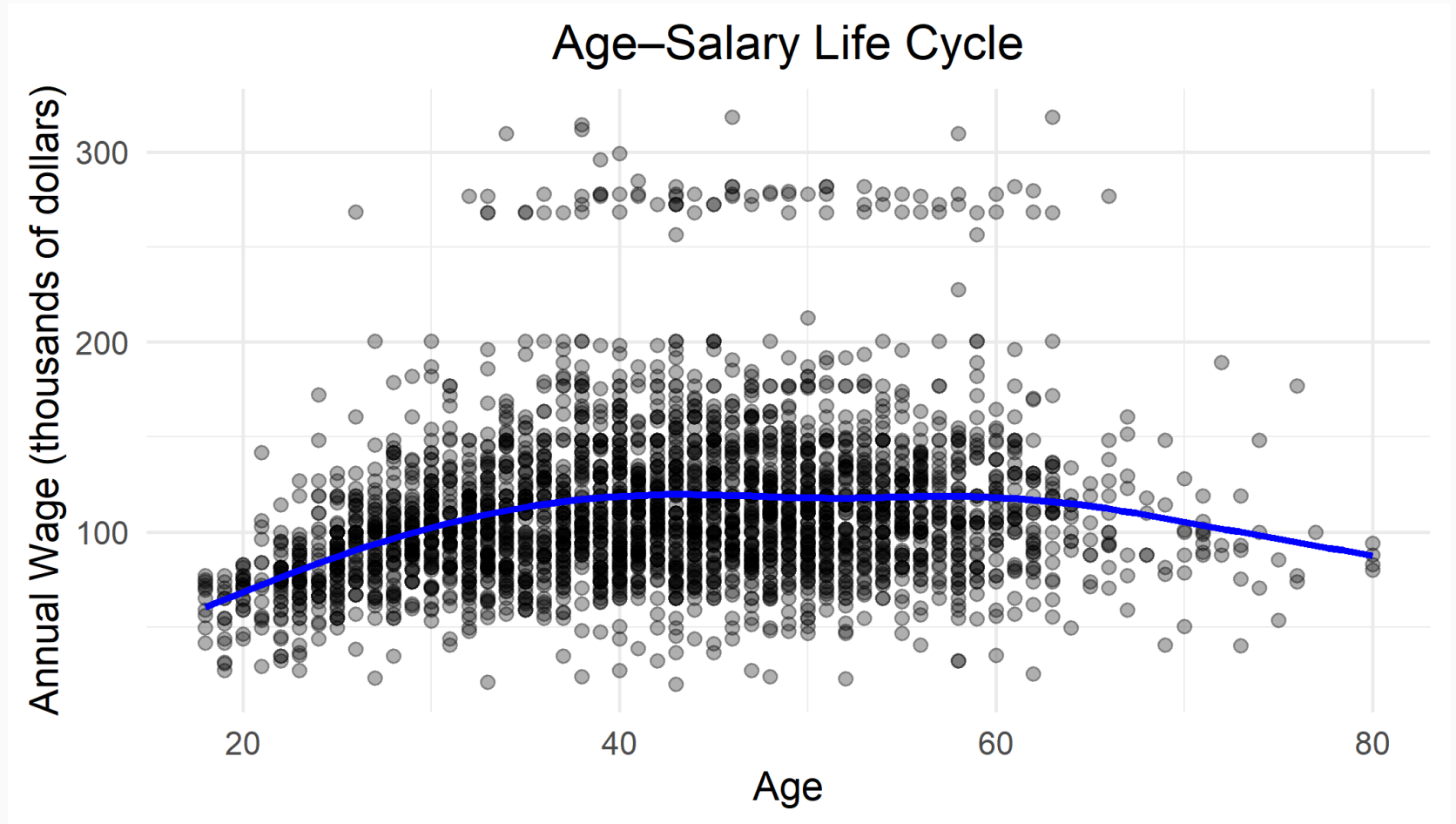


A Simple Decision Tree



# Example: Salary vs. Age

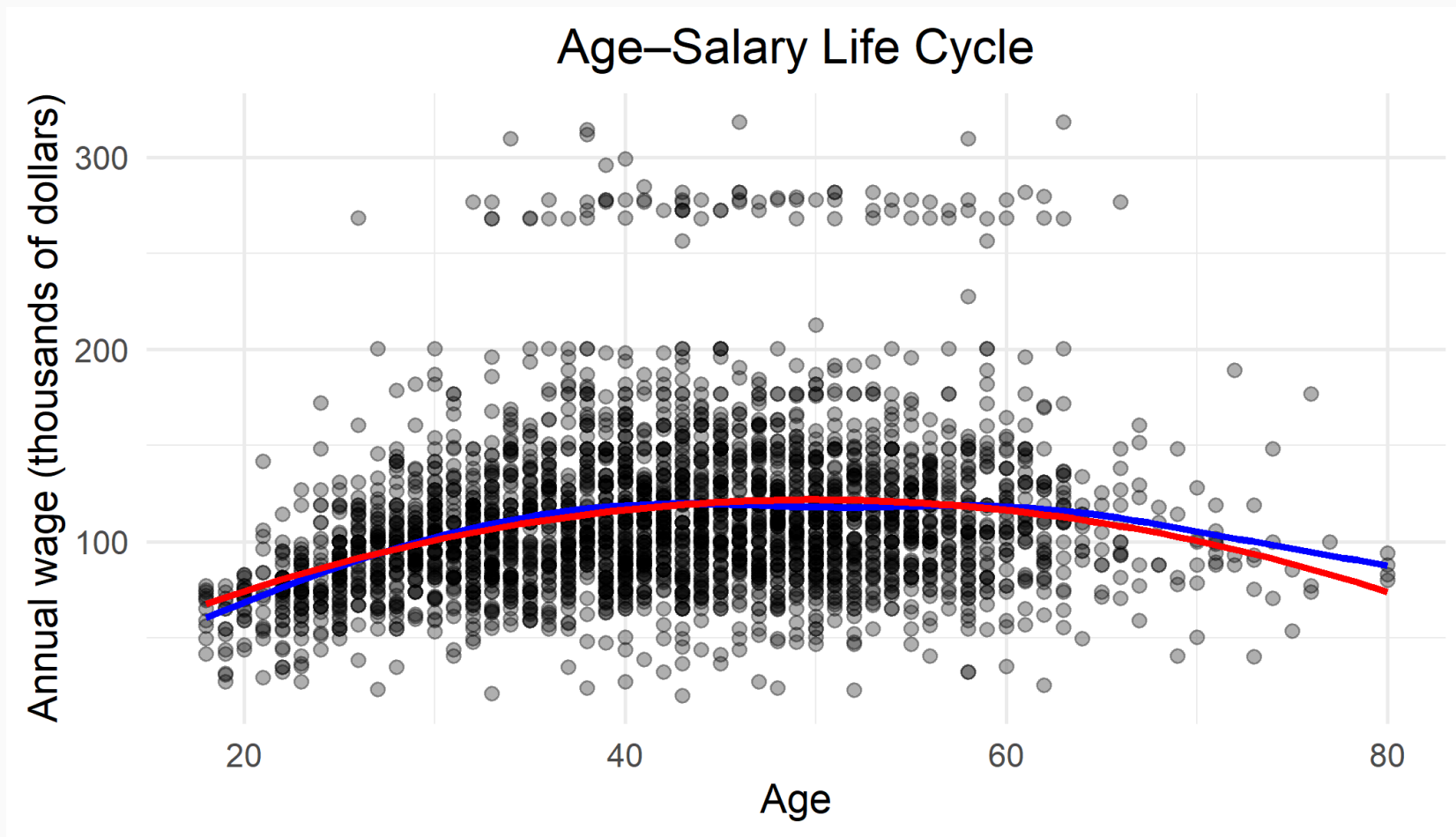
Let's demonstrate with the `wage` dataset.





# Quadratic?

We can fit a quadratic curve, but it does not seem entirely right.



# Piece-wise Linear Approach

Instead of fitting a single quadratic curve, it may be more natural to think of a **career in three stages**:

1. **Early Career (Learning & Skill Accumulation)** Wages rise as age and experience increase.
2. **Mid-Career (Established)** Wages plateau as growth stabilizes.
3. **Retirement** Wages decline or level off at retirement income.

One way to visualize this is to **split the graph into three regions** and fit a **separate line for each stage**.

**Question:** How should we determine the **breakpoints** between these stages with data?

# Grow a Tree

When you plant a real tree, it grows gradually toward the sunlight.

Our decision tree grows in a similar step-by-step fashion:

- Find the **first split** that gives the largest reduction in the loss function.
- Then, within each resulting region, find the **next best** split.
- Continue splitting **sequentially**, each time choosing the split that reduces the loss the most.

Over time, this produces a full, reasonably large tree.

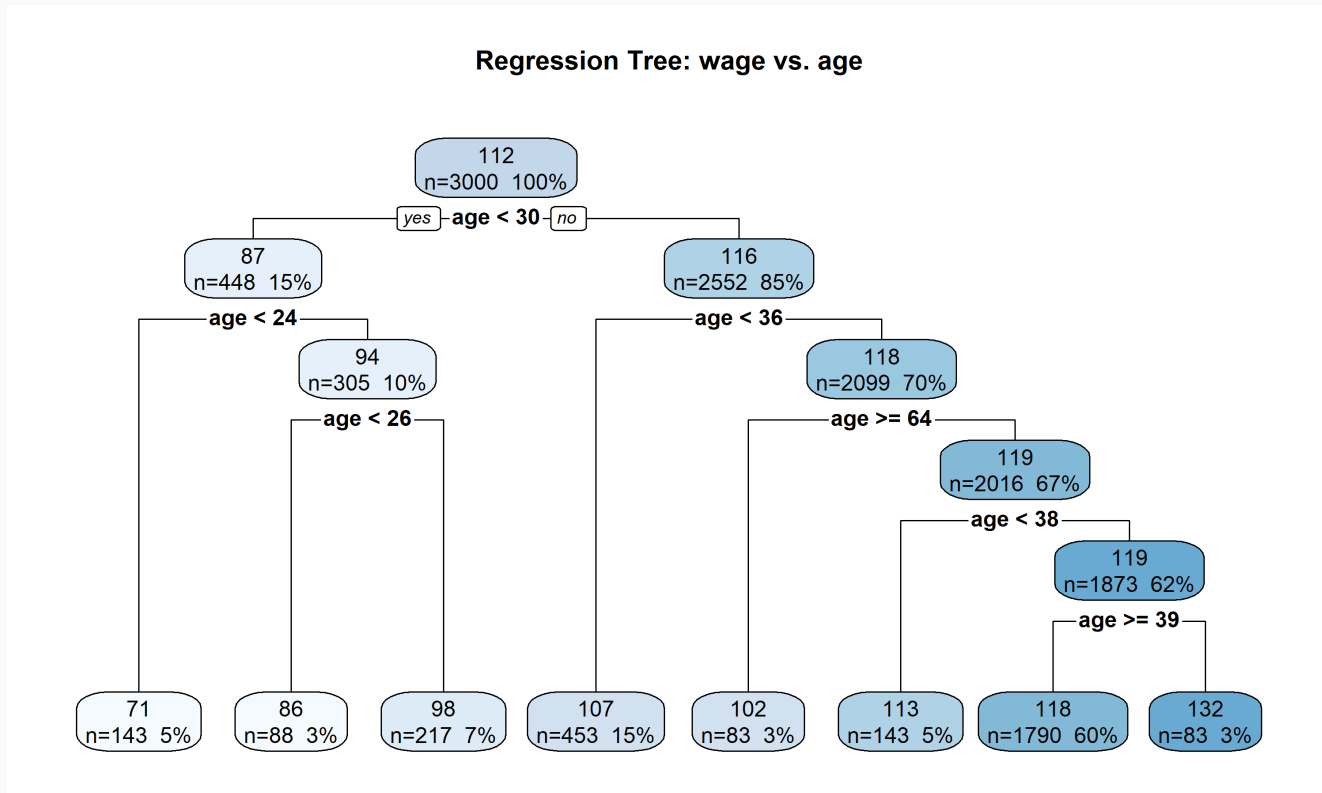
The core idea of tree-based methods is to **group similar observations together**.

Each split tries to create regions where observations look as similar as possible (same outcome value, or following the same small model).

By the time we reach the leaves, each group is internally homogeneous.

# Regression Tree

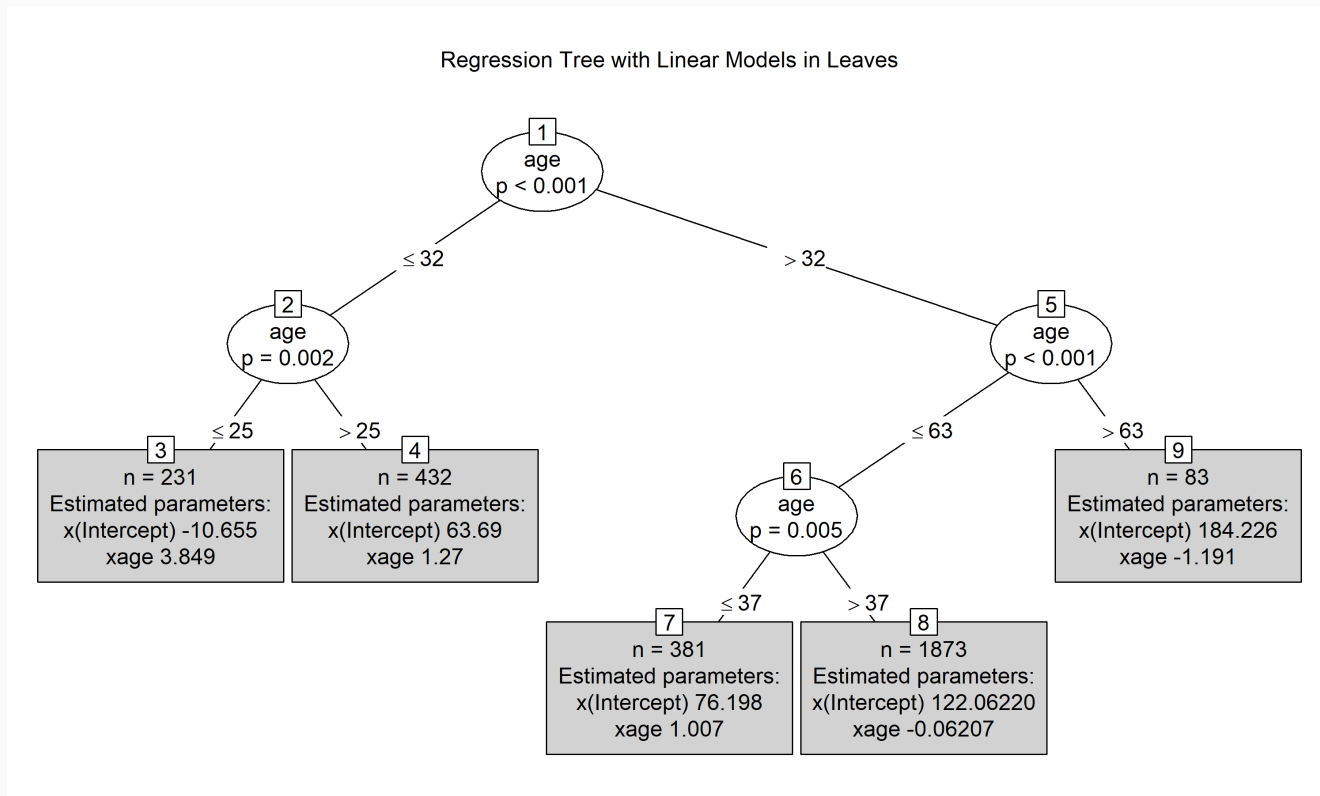
We can directly "grow" a regression tree: One value for each leaf.



# Regression Tree with Linear Leaf

However, each segment shows a clear trend -> fit a linear model in each leaf!

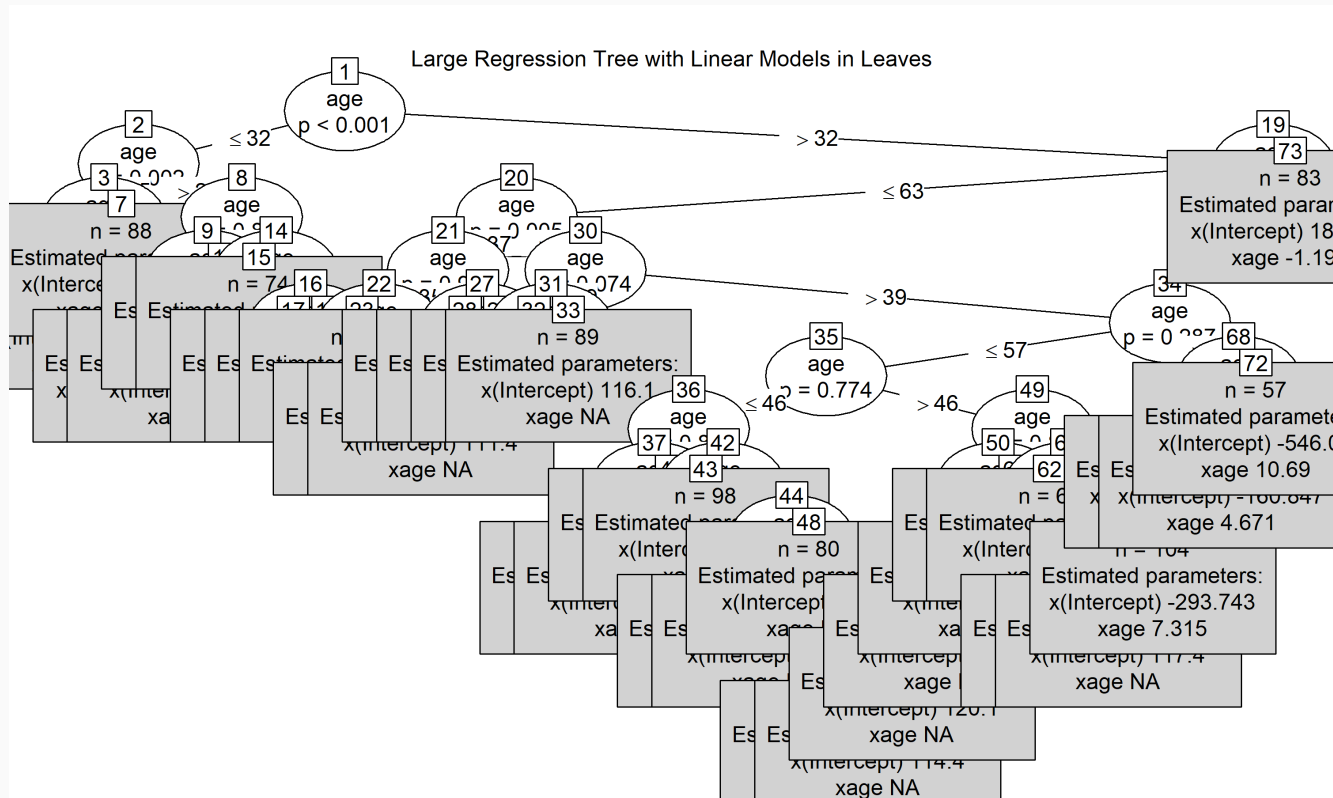
This produces a much simpler tree with fewer leaves.



Does the result align with our intuition?

# A Big Tree?

The default tree is relatively small and fit our intuition. Alternatively, we can grow a very large tree:



# Pruning: Big vs. Small

Usually, we want the tree to stay **relatively small**-this makes it easier to interpret and helps avoid **overfitting**.

Suppose we grow a regression tree by minimizing the **sum of squared residuals (SSR)**, and let  $|T|$  denote the number of terminal nodes (leaves). Our goal is to achieve a low SSR **while keeping the tree as simple as possible**.

How do we balance fit vs. complexity?

This is a classical problem of **bias-variance trade-off**: Small trees are biased, but with less variance!

Suppose we are growing a tree to minimize Sum Squared Residual (SSR) and  $|T|$  is the number of terminal nodes (leaves). We are trying to minimize SSR will having as little leaves/splits as possible. How should we do that?

# Pruning

It works like LASSO!

We solve:

$$\min \sum_i (y_i - \hat{y}_i) + \alpha |T|$$

The penalty term  $\alpha |T|$  discourages excessive splitting.

- A large  $\alpha$  -> simpler tree (fewer leaves).
- A small  $\alpha$  -> more complex tree.



# Pruning

Tuning parameters that control tree complexity:

- "cp" in `rpart`
- "alpha" in `partykit`

These are the most important hyperparameters for controlling tree size.

Internally, both packages also include additional stopping rules (pre-pruning) to prevent the tree from “growing into madness.” See `?mob_control` for details.

Another strategy is to first grow a large tree, then prune unnecessary branches afterward (Post-pruning).

In practice, one often uses **cross-validation** or a **validation set** to select the optimal tree size or tuning parameter. For `partykit`, manual implementation with for loop is needed. However, this is internal in `rpart`.

# Cross-validation with Trees

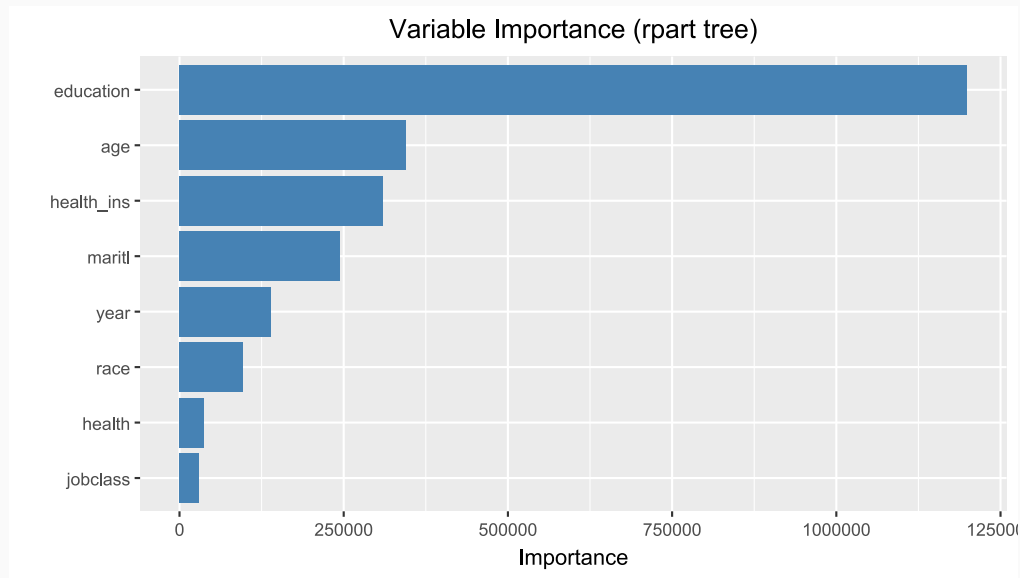
Here is the result of `rpart` cp value information. `xerror` is the cross-validation error:

```
##
## Regression tree:
## rpart(formula = wage ~ age, data = Wage, method = "anova", control = rpart.control(cp =
##
## Variables actually used in tree construction:
## [1] age
##
## Root node error: 5222086/3000 = 1740.7
##
## n= 3000
##
##          CP nsplit rel error  xerror    xstd
## 1 0.0629073      0  1.00000 1.00116 0.047777
## 2 0.0104881      1  0.93709 0.94625 0.045947
## 3 0.0083518      2  0.92660 0.93422 0.045520
## 4 0.0042840      3  0.91825 0.93259 0.045509
## 5 0.0018358      4  0.91397 0.92753 0.045216
## 6 0.0018145      6  0.91030 0.92524 0.045117
## 7 0.0010000      7  0.90848 0.92252 0.044969

## [1] "The best value for cp is: 0.001"
```

# Tree with more Variables

We can add more variables to the tree. Suppose now, we are using all variables to predict wage in the `rpart` regression tree. It is harder to interpret, so we have this **variable importance plot**:



# Test a model

```
# Fit a tree from training data (For demonstration, you can change cp to 0.0001)
fit2_RegTree <- rpart(wage ~ •-logwage, data = Wage_train, method = "anova",
                      control = rpart.control(cp = 0.001))
# printcp(fit2_RegTree)      # prints CV table

# You then can use the following code to extract the best model (via 10-fold CV)
opt_index <- which.min(fit2_RegTree$cptable[, "xerror"])
best_cp <- fit2_RegTree$cptable[opt_index, "CP"]
# paste0("The best value for cp is: ", round(best_cp, 5))

# Refit the model
rpart_tree2 <- rpart(wage ~ •-logwage, data = Wage_train, method = "anova",
                    control = rpart.control(cp = best_cp))

# rpart_tree2

# predict with test data
pred <- predict(rpart_tree2, newdata = Wage_test)

# estimate the OOS MSE
oos_mse <- mean((Wage_test$wage - pred)^2)
paste0("The OOS MSE is: ", round(oos_mse, 2))

## [1] "The OOS MSE is: 1307.75"
```

# Bagging, Boosting and Random Forest

---

# Bagging: An Ensemble Learning Idea

Bagging (Bootstrap Aggregating) uses the power of **many models combined together**.

We repeatedly draw **bootstrap samples** from the training data (sampling *with replacement*), and fit a separate **large** tree to each sample.

This is not for computing standard errors — it is used purely for **improving prediction**.

The final prediction is simply the average of all trees:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

By averaging many noisy trees (from CLT):

- **Variance decreases** (major benefit)
- **Bias increases slightly** (usually small)
- **Overall prediction accuracy improves** (at the cost of interpretability)

**Ensemble** is a powerful idea that extends far beyond trees! It can usually drastically improve the prediction performance.

# Random Forest

Random Forest builds on the idea of bagging and it is a nice improvement of Bagging: in each bootstrap, it randomly select  $m$  predictors. for tree fitting, given total  $p$  predictors. (Usually  $m = \sqrt{p}$ .  $m = p$  would be back to bagging!)

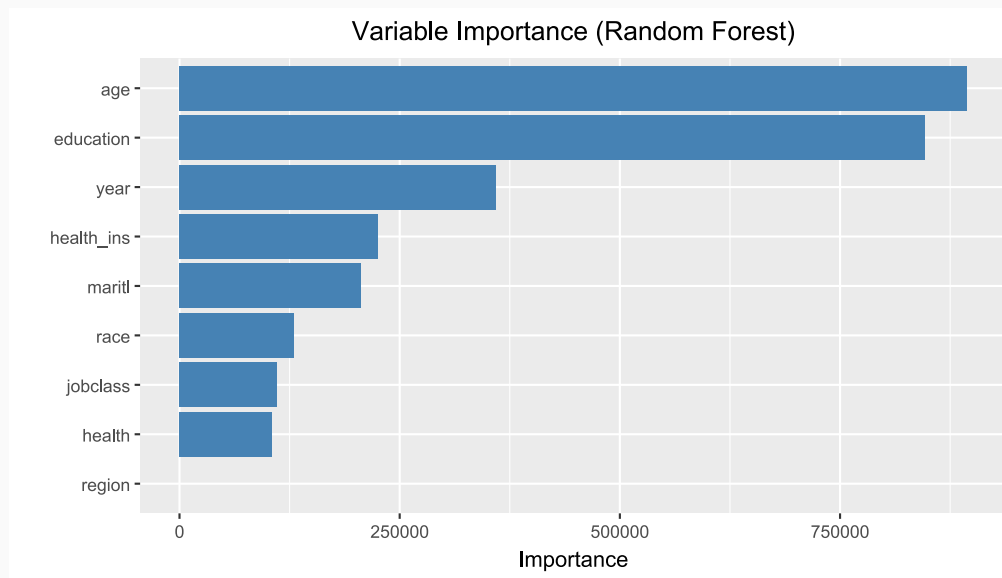
With this random selection, the trees from each bootstrap are less correlated so variance can decrease further!

The idea is that each variable has equal chance to be on each tree. A valid predictor will have a higher chance to be in the model splits! By aggregating, hopefully, the invalid predictors get averaged out.

We use a modern package of Random Forest, `ranger`. There are two places to `set.seed!`

For Random Forest or Bagging, we usually have the K-fold CV for free. This is called the **Out-of-Bag error (OOB)**.

```
## [1] "OOB prediction error MSE: 1215.48054503941"
```





# CV with caret

We can use `caret` in Lecture 10 to perform k-fold CV for Random Forest. "mtry" is the main hyperparameter here:

```
## Random Forest
##
## 2100 samples
##   10 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 1680, 1680, 1679, 1681, 1680
## Resampling results across tuning parameters:
##
##   mtry  splitrule  RMSE      Rsquared  MAE
##   2     variance  36.31967  0.3448557  25.00670
##   2     extratrees 36.57206  0.3383352  25.23230
##   13    variance  34.95984  0.3027713  24.19750
##   13    extratrees 34.19922  0.3246298  23.68524
##   24    variance  36.20787  0.2725617  25.21992
##   24    extratrees 35.39748  0.2936230  24.70889
##
## Tuning parameter 'min.node.size' was held constant at a value of 5
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were mtry = 13, splitrule = extratrees
```

# Boosting

Boosting is also using bootstrap. But unlike averaging a bunch of **large** trees like Bagging or Random Forest, boosting sequentially grows the later tree based on the previous tree:

1. Starting from a empty/null model  $\hat{f}(x) = 0$  and residuals  $e_i = y_i$  for all  $i$  in the training set
2. For  $b = 1, 2, \dots, B$ , repeat:
  - Fit a **small** tree  $\hat{f}^b$  with  $d$  splits ( $d + 1$  leaves) to the training data  $(X, e)$ . (Using  $X$  to fit the residuals.)
  - Update  $\hat{f}$  by adding in a shrunk version of the new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \eta \hat{f}^b(x).$$

- Update the residuals,

$$e_i \leftarrow e_i - \eta \hat{f}^b(x_i).$$

3. Output the boosted model,

$$\hat{f}(x) = \eta \sum_{b=1}^B \hat{f}^b(x).$$

# Hyperparameter

$B$ ,  $d$  and  $\eta$  are the hyperparameters:

- Usually,  $\eta = 0.01$  or  $0.001$
- Often,  $d = 1$  works well
- Use CV to select  $B$ .

$\eta$  is making machine forget part of what it has learned. By slow learning, the performance gets better!

We use a modern package of boosting, `xgboost`. There are two places to `set.seed`!

```
library(xgboost)
```

```
# Set the model
```

```
X ← model.matrix(~ . - wage - logwage - 1, data = Wage_train) # converts all factors to numeric  
y ← Wage_train$wage
```

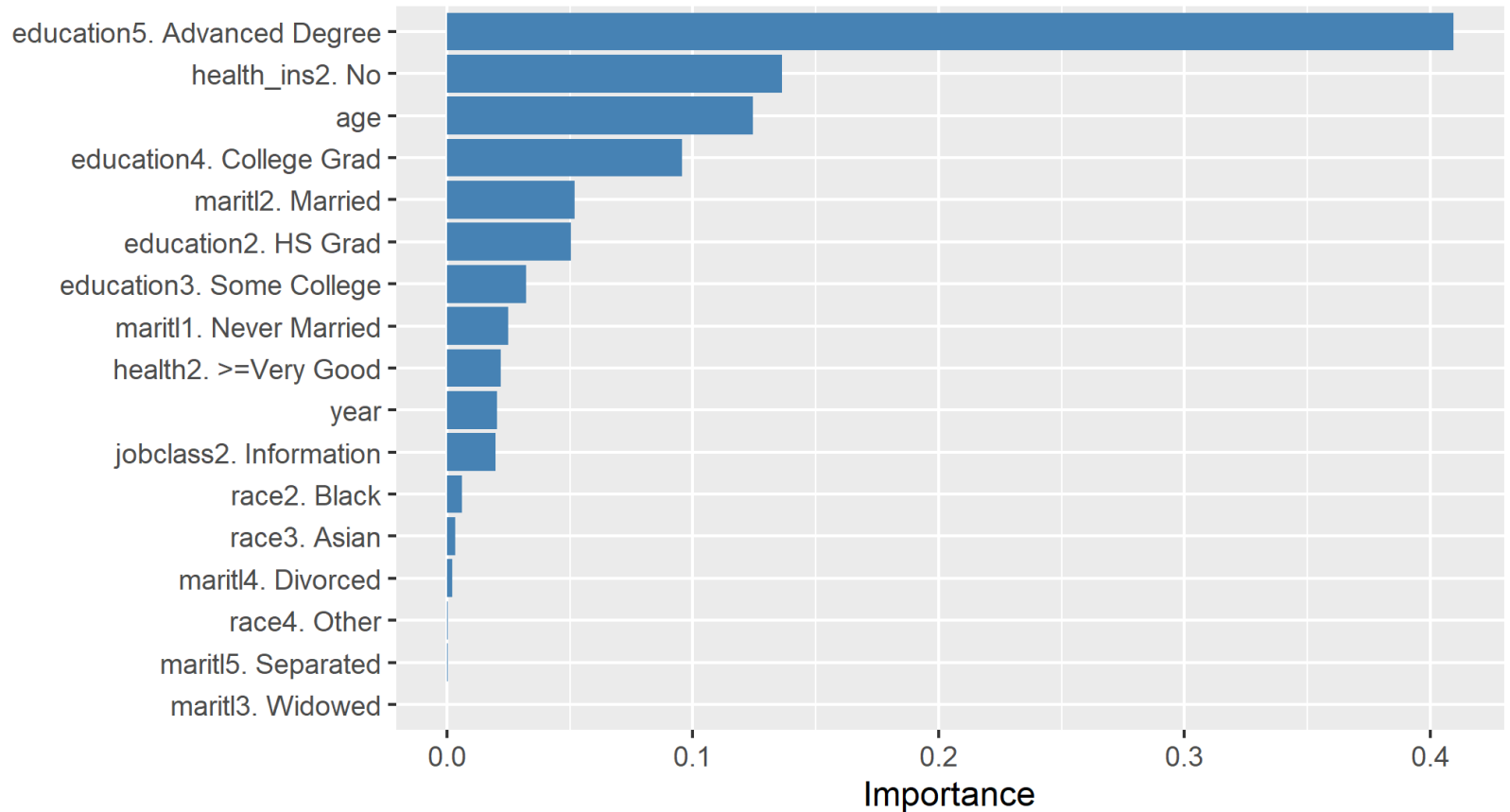
```
# First seed
```

```
set.seed(123)
```

```
xgb_model ← xgboost(  
  data = X,  
  label = y,  
  objective = "reg:squarederror",  
  nrounds = 200,  
  eta = 0.01,  
  max_depth = 4,  
  subsample = 0.8,  
  colsample_bytree = 0.8,  
  verbose = 0, # silence the fitting message  
  seed = 123 # main seed  
)
```

# Variable Importance

Variable Importance (XGBoost)



# CV + Early Stopping for Boosting

**Early stopping** prevents wasting iterations on trees that don't improve performance.

For example, if `early_stopping_rounds = 50`, the training stops if the out-of-sample error does not improve for 50 consecutive rounds.

```
set.seed(123)
# 5-fold CV for boosting
cv_res <- xgb.cv(
  data = X,
  label = y,
  nrounds = 2000,
  eta = 0.01,
  nfold = 5,
  objective = "reg:squarederror",
  early_stopping_rounds = 50,
  verbose = 0
)
# Find the best number of iterations
cv_res$evaluation_log[cv_res$best_iteration,]
```

```
##      iter train_rmse_mean train_rmse_std test_rmse_mean test_rmse_std
##      <num>          <num>          <num>          <num>          <num>
## 1:    352          27.96055          0.4190022          34.62349          1.222707
```

# Model Evaluations

```
## [1] "The mse of the pruned tree is: 1307.75"
```

```
## [1] "The mse of the Random Forest model is: 1259.66"
```

```
## [1] "The mse of the Boosting model is: 1347.05"
```

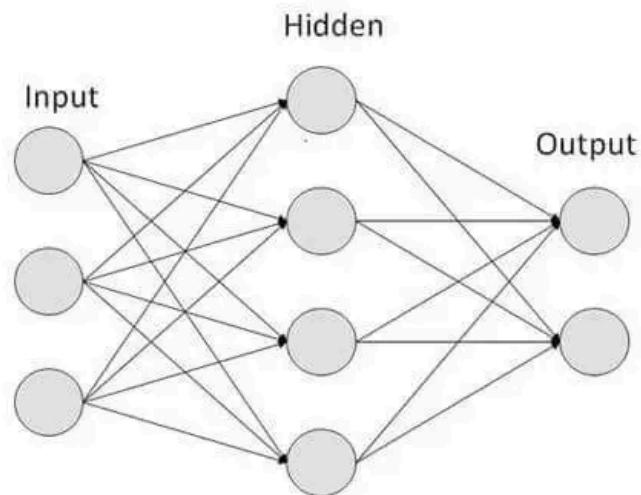
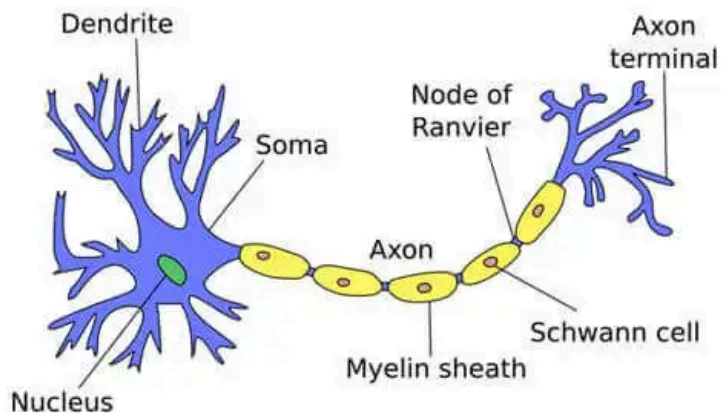
These results are not universal. Different methods suit different data.

# Basic Neural Networks with Torch

---



# Why called Neural Networks?



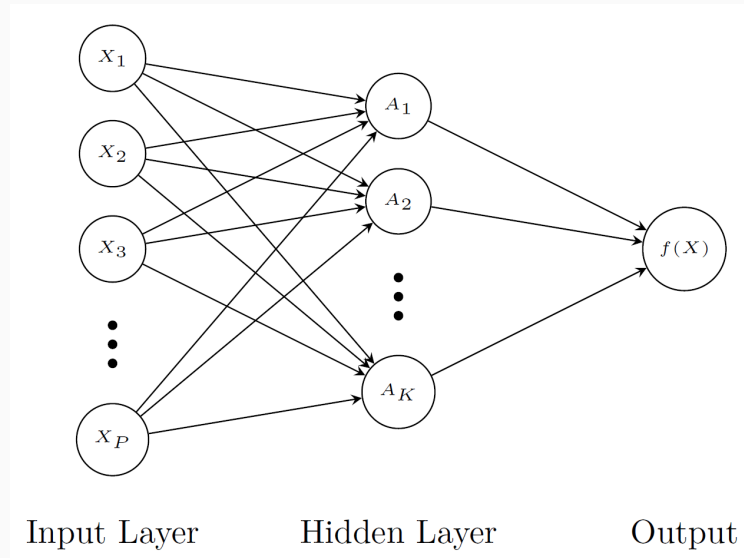
Here are youtube videos about: [Neurons in brain](#) and [How neurons make connections](#).

Image from [this](#) blog about neural networks.

There is an archived folder on Canvas about setting up `keras`, but it's complicated and relies on Python. Instead, we will use `torch`, which is much easier to set up and does not

# Single Layer Nerual Network

The neural networks usually is described using network diagram:



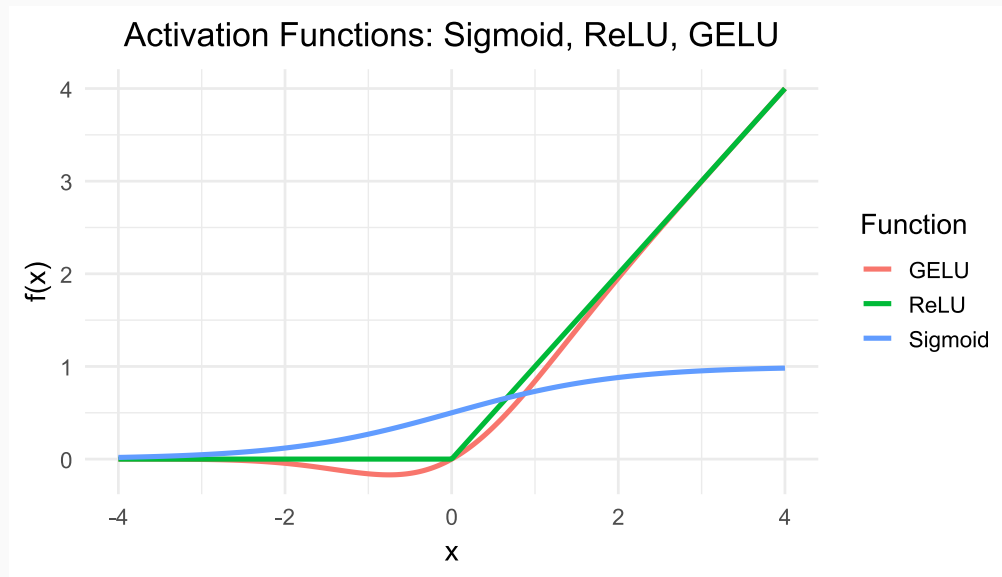
$$A_k = g\left(\sum_{j=1}^P w_{jk}^0 x_j + b_k^0\right)$$

$$f(X) = \sum_{k=1}^P w_k^1 A_k + b^1$$

# Activation Function

The function  $g$  is called **activation function**. Some popular activation functions are:

- Sigmoid function
- ReLu (Rectified Linear Unit) function
- GELU function



We will use *ReLU* in our examples.

# Fitting a Neural Network

- Goal remains:

$$\min \sum_{i=1}^N (y_i - f(x_i))^2.$$

- Trained with **Stochastic Gradient Descent (SGD)**
  - Uses minibatches (random subsets)
  - Moves "downhill" using derivatives
  - **Backpropagation = Chain Rule** for efficient gradient computation
- **Key hyperparameters:** learning rate, number of epochs
- **Regularization:**
  - Ridge, LASSO
  - Early stopping, dropout (randomly omitting variables like RF), data augmentation (move X slightly around)
- **Stochastic nature:** updates parameters using minibatches -> adds randomness and speeds computation

# Wage NN

Let's fit a one-layer Neural Network for Wage:

```
## An `nn_module` containing 1,729 parameters.
```

```
##
```

```
## — Modules
```

---

```
## • hidden: <nn_linear> #1,664 parameters
```

```
## • activation: <nn_relu> #0 parameters
```

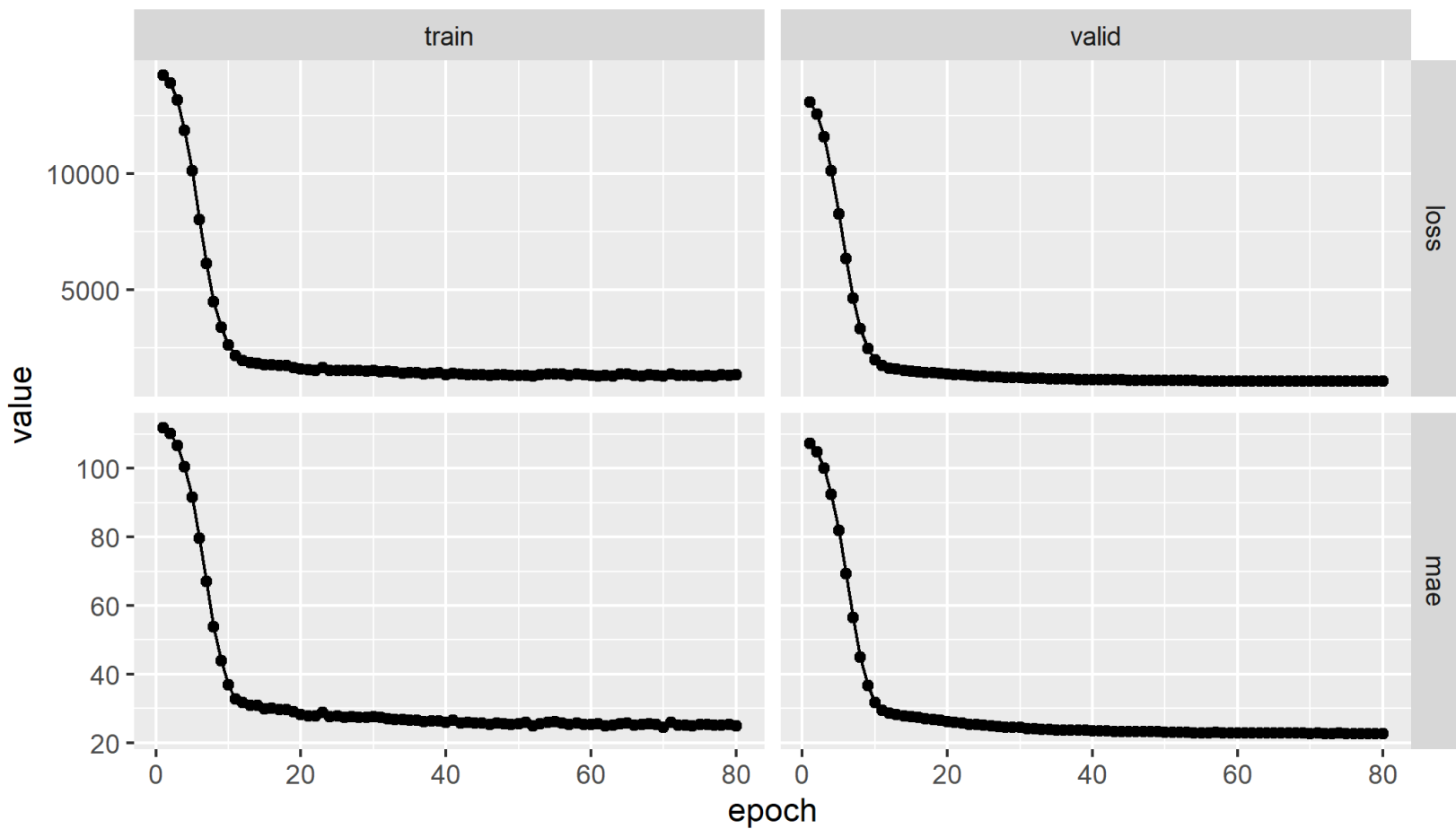
```
## • dropout: <nn_dropout> #0 parameters
```

```
## • output: <nn_linear> #65 parameters
```

The number of parameters are huge even for one layer!

Modern LLMs all employ Deep Neural Networks, so you can imagine how many parameters there are and how costly the training are. E.g. GPT-5 (high) is estimated to have 635B parameters.

# Result



# OOS Performance

```
# Find the predicted with fitted model  
pred_test ← predict(fitted, x_test) ▷  
  as.numeric()
```

```
# find the mse of neural networks  
paste0("The mse of the Neural Networks model is: ", round(mean((y_test - as.numeric(p
```

```
## [1] "The mse of the Neural Networks model is: 1278.29"
```

Here are some more NN resources with `torch`:

- [Here](#) is the announcement of `torch` in R.
- `torch` package information
- Deep Learning and Scientific Computing with R `torch`
- Large Language Model tools for R

# Torch Applications and GenAI

---



# Examples

I have posted all the example code in [torch\\_applications folder](#). Some are useful packages other than `torch`. We will go over them one by one:

- MINST Digits: `Torch_MINST.Rmd`
- Small LLM Training: `Torch_LLM_Training.R`
- Small LLM Generating: `Torch_LLM_Generating.Rmd`
- Image Recognition: `Torch_Image_Recognition.Rmd`
- Sentiment Analysis: `Torch_Sentiment.Rmd`
- Local Ollamar in R: `Ollamar_Language_Model.Rmd`

I hope this class has been a valuable learning experience and will contribute meaningfully to your bright future!

---